🐍 PYTHON

Complete Handwritten-Style Notes

Beginner se Advance tak — Diagrams, Examples & Code ke saath

Based on the roadmap.sh Python Roadmap

📑 Table of Contents

🐍 Python Complete Notes — Beginner se Advance tak

Ye notes roadmap.sh ke Python Roadmap par based hain. Isme beginner level ki basic syntax se lekar advance level ke frameworks, concurrency aur tooling tak sab kuch cover kiya gaya hai. Har topic me: simple explanation, diagram/flow, code example aur important note diya gaya hai.

> **Kaise padhein:** Topics ko order me padhein (upar se niche). Ek topic khatam karke code example khud type karke run karein — isse concept pakka clear hoga.

📌 Roadmap Overview (Diagram)

Stage	Topics
1. Basics	Syntax, Variables, Conditionals, Casting, Exceptions, Functions
2. Collections	Lists, Tuples, Sets, Dictionaries, Loops
3. DSA	Arrays, Linked List, Stack/Queue, Hash Table, BST, Recursion, Sorting
4. Core Python Pro	Modules, Lambda, Decorators, Iterators, Regex, OOP
5. Tooling	Package Managers, Environments, Static Typing, Formatting, Docs, Testing
6. Advanced	Frameworks, Concurrency, Asynchrony

1 Basics — Foundation of Python

1.1 Basic Syntax

Python ek high-level, interpreted, aur beginner-friendly programming language hai. Indentation (spaces) se blocks define hote hain — curly braces {} ki zaroorat nahi hoti.

```
# Ye ek comment hai
print("Hello, World!")   # simplest program

if True:
    print("Indentation important hai")   # 4 spaces indent
```

Tip: Python me semicolon (;) optional hai, aur indentation consistent honi chahiye (4 spaces standard hai).

1.2 Variables and Data Types

Variable ek naam hai jo kisi value ko store karta hai. Python dynamically typed hai — type declare karne ki zaroorat nahi, value assign karte hi type decide ho jata hai.

Data Type	Example	Description
int	x = 10	Whole number
float	x = 10.5	Decimal number
str	x = "hi"	Text / string
bool	x = True	True / False
complex	x = 2+3j	Complex number

```
name = "Rahul"      # str
age = 21             # int
height = 5.8         # float
is_student = True    # bool
print(type(age))     # <class 'int'>
```

1.3 Conditionals

Conditionals se hum decision leke code ko branch karte hain — if, elif, else.

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Flow	
if condition	-> True hone par block run
elif condition	-> pehli condition False, ye check
else	-> jab sab conditions False hon

1.4 Type Casting

Type casting ka matlab hai ek data type ko dusre type me convert karna.

```
x = "10"
y = int(x)     # str -> int
z = float(y)   # int -> float
s = str(z)     # float -> str
print(y, z, s)
```

> **Implicit vs Explicit:** Implicit casting Python khud karta hai (int + float = float), Explicit casting hum int(), str(), float() se manually karte hain.

1.5 Exceptions (Error Handling)

Jab program run time par error de, use exception kehte hain. try-except se hum errors ko handle karke program crash hone se bachate hain.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("Zero se divide nahi ho sakta!")
except ValueError:
    print("Sahi number daalein!")
else:
    print("Result:", result)
finally:
    print("Program complete hua")
```

Block	Kaam
try	Risky code yahan likhte hain
except	Error aane par ye chalta hai
else	Agar koi error nahi aayi to ye chalta hai
finally	Hamesha chalta hai (error ho ya na ho)

1.6 Functions & Builtin Functions

Function code ka reusable block hota hai. def keyword se function banate hain.

```
def greet(name, greeting="Hello"):
    """Ye function greeting return karta hai"""
    return f"{greeting}, {name}!"

print(greet("Amit"))
print(greet("Sara", "Namaste"))
```

Builtin functions wo functions hote hain jo Python me already available hote hain, import karne ki zaroorat nahi:

```
print(len("hello"))     # 5
print(max(3, 7, 2))     # 7
print(sorted([3,1,2]))  # [1, 2, 3]
print(sum([1,2,3]))     # 6
print(type(10))         # <class 'int'>
```

Yaad rakhein: *args aur **kwargs se function me variable number ke arguments pass kar sakte hain: def f(*args, **kwargs):

2 Collections — Lists, Tuples, Sets, Dictionaries, Loops

2.1 Lists

List ek ordered aur mutable (changeable) collection hai. Square brackets [] me likhte hain.

```
fruits = ["apple", "banana", "mango"]
fruits.append("orange")     # add karna
fruits[0] = "grape"          # update karna
fruits.remove("banana")      # delete karna
print(fruits[0:2])           # slicing -> first 2 items
print(len(fruits))
```

2.2 Tuples

Tuple bhi ordered collection hai lekin immutable (ek baar bana to change nahi ho sakta). Round brackets () me likhte hain.

```
point = (10, 20)
x, y = point       # unpacking
print(x, y)
# point[0] = 5    # Error! tuple change nahi ho sakta
```

2.3 Sets

Set ek unordered collection hai jisme duplicate values allowed nahi. Curly braces {} me likhte hain.

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)   # union -> {1,2,3,4,5}
print(a & b)   # intersection -> {3}
print(a - b)   # difference -> {1,2}
```

2.4 Dictionaries

Dictionary key-value pairs store karti hai — jaise ek real dictionary me word aur uska meaning hota hai.

```
student = {"name": "Riya", "age": 20, "city": "Jaipur"}
print(student["name"])       # value access
student["age"] = 21          # update
student["grade"] = "A"       # naya key-value add
for key, value in student.items():
    print(key, "->", value)
```

2.5 Loops

Loops se hum same code ko baar-baar repeat karte hain — for loop aur while loop.

```
# for loop
for i in range(5):
    print(i)          # 0 1 2 3 4

# while loop
n = 0
while n < 3:
    print("n =", n)
    n += 1
```

```
# break aur continue
for i in range(10):
    if i == 5:
        break         # loop ruk jayega
    if i % 2 == 0:
        continue      # is iteration ko skip karega
    print(i)
```

> **List vs Tuple vs Set vs Dict:** List = ordered+mutable | Tuple = ordered+immutable | Set = unordered+unique | Dict = key-value pairs

3 Data Structures & Algorithms (DSA)

DSA ka matlab hai data ko efficiently store karna (Data Structures) aur usko efficiently process karna (Algorithms).

3.1 Arrays and Linked Lists

Array ek fixed sequence hoti hai jisme elements memory me continuous jagah par store hote hain (Python me list is kaam ke liye use hoti hai). Linked List me har element (node) apne agle element ka address (pointer) rakhta hai.

Array	[10] [20] [30] [40] — index se direct access (fast)
Linked List	[10|next] -> [20|next] -> [30|next] -> None

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def display(self):
        curr = self.head
        while curr:
            print(curr.data, end=" -> ")
            curr = curr.next
        print("None")
```

3.2 Stacks, Queues, Heaps

Structure	Order	Real life example
Stack	LIFO (Last In First Out)	Plates ka stack
Queue	FIFO (First In First Out)	Ticket line
Heap	Priority based (min/max)	Hospital emergency priority

```
# Stack (Python list se)
stack = []
stack.append(1)   # push
stack.append(2)
stack.pop()        # pop -> last element remove

# Queue (collections.deque se)
from collections import deque
```

```
queue = deque()
queue.append(1)        # enqueue
queue.append(2)
queue.popleft()         # dequeue -> first element remove

# Heap (heapq module)
import heapq
nums = [5, 1, 8, 3]
heapq.heapify(nums)
print(heapq.heappop(nums))   # smallest element pehle
```

3.3 Hash Tables

Hash Table key ko hash function se ek index me convert karke value store karti hai — bahut fast lookup (O(1) average). Python me dictionary internally hash table hi hai.

```
phone_book = {}
phone_book["Amit"] = "9876543210"
phone_book["Neha"] = "9123456789"
print(phone_book["Amit"])   # O(1) fast lookup
```

3.4 Binary Search Tree (BST)

BST ek tree structure hai jisme har node ke left side me usse chhoti values aur right side me badi values hoti hain.

```
        8
      /   \
     3     10
    / \      \
   1   6      14
```

```
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if root is None:
        return Node(val)
    if val < root.val:
        root.left = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root
```

3.5 Recursion

Recursion me function khud ko call karta hai jab tak base condition true na ho jaye.

```
def factorial(n):
    if n == 0:              # base case
        return 1
    return n * factorial(n - 1)   # recursive call

print(factorial(5))    # 120
```

> **Dhyan rahe:** Har recursive function me ek base case zaroor hona chahiye, warna infinite recursion ho jayega (RecursionError).

3.6 Sorting Algorithms

Algorithm	Time Complexity	Idea
Bubble Sort	O(n²)	Baar baar adjacent elements ko compare-swap karo
Selection Sort	O(n²)	Smallest dhundo, front me rakho
Insertion Sort	O(n²)	Ek-ek element ko sahi jagah insert karo
Merge Sort	O(n log n)	Divide aur merge (divide & conquer)
Quick Sort	O(n log n) avg	Pivot chuno, chhote/bade me baato

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]   # swap
    return arr

print(bubble_sort([5, 2, 9, 1, 5]))   # [1, 2, 5, 5, 9]

# Python builtin sort (Timsort algorithm use karta hai)
print(sorted([5, 2, 9, 1, 5]))
```

4 Core Python — Pro Level Concepts

4.1 Modules

Module ek .py file hoti hai jisme functions, classes, variables likhe hote hain, jinhe hum dusri files me import karke reuse karte hain.

```
# math_utils.py
def add(a, b):
    return a + b

# main.py
import math_utils
print(math_utils.add(2, 3))

from math_utils import add   # sirf ek function import
import math                    # builtin module
print(math.sqrt(16))
```

4.2 Lambdas

Lambda ek chhota, anonymous (bina naam ka) function hota hai, jo ek line me likha jata hai.

```
square = lambda x: x * x
print(square(5))     # 25

nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))
print(doubled)       # [2, 4, 6, 8]
```

4.3 Decorators

Decorator ek function hai jo dusre function ko 'wrap' karke uska behaviour badalta hai, bina uska original code change kiye.

```
def my_decorator(func):
    def wrapper():
        print("Function chalne se pehle")
        func()
        print("Function chalne ke baad")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
# Output:
# Function chalne se pehle
# Hello!
# Function chalne ke baad
```

4.4 Iterators

Iterator wo object hai jiske paas __iter__() aur __next__() methods hote hain, jisse hum values ko one-by-one nikal sakte hain.

```
class Counter:
    def __init__(self, limit):
        self.limit = limit
        self.num = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.num < self.limit:
            self.num += 1
            return self.num
        raise StopIteration

for n in Counter(3):
    print(n)     # 1 2 3
```

4.5 Regular Expressions (Regex)

Regex ek pattern-matching tool hai jisse hum text ke andar patterns dhundte hain (jaise email, phone number validate karna).

```
import re

text = "Contact: amit123@gmail.com"
match = re.search(r"[\w.]+@[\w.]+", text)
print(match.group())    # amit123@gmail.com

print(re.findall(r"\d+", "I have 2 cats and 3 dogs"))   # ['2', '3']
```

Symbol	Meaning
\d	koi bhi digit (0-9)
\w	word character (a-z, A-Z, 0-9, _)
+	1 ya usse zyada baar
*	0 ya usse zyada baar
.	koi bhi single character

4.6 Object Oriented Programming (OOP)

OOP ek programming style hai jisme code ko 'objects' ke roop me organize karte hain — real world entities ki tarah.

Classes

```
class Car:
    def __init__(self, brand, speed):   # constructor
        self.brand = brand
        self.speed = speed

    def drive(self):                      # method
        print(f"{self.brand} chal rahi hai {self.speed} km/h par")
```

```
my_car = Car("Tesla", 120)
my_car.drive()
```

Inheritance

Inheritance se ek class (child) dusri class (parent) ke properties aur methods use kar sakti hai.

```
class Vehicle:                # parent class
    def start(self):
        print("Vehicle start ho gayi")

class Bike(Vehicle):          # child class
    def start(self):
        super().start()       # parent ka method call
        print("Bike bhi start ho gayi")

Bike().start()
```

Methods & Dunder Methods

Dunder (double underscore) methods jaise __init__, __str__, __len__ Python ki special/magic methods hain jo built-in operators ko customize karti hain.

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):        # print() par ye chalta hai
        return f"Book: {self.title}"

    def __len__(self):        # len() par ye chalta hai
        return self.pages

b = Book("Python 101", 300)
print(b)          # Book: Python 101
print(len(b))     # 300
```

OOP Pillar	Matlab
Encapsulation	Data aur methods ko ek class me bandhna
Inheritance	Ek class dusri class ke features use kare
Polymorphism	Same method, alag classes me alag behaviour
Abstraction	Complex implementation chhupana, sirf zaroori dikhana

4.7 List Comprehensions & Generator Expressions

List Comprehension ek short syntax hai list banane ka. Generator Expression bhi same dikhta hai lekin values ek-ek karke (lazy) generate karta hai, memory bachata hai.

```
# List comprehension
squares = [x*x for x in range(5)]
```

```
print(squares)          # [0, 1, 4, 9, 16]

# Condition ke saath
evens = [x for x in range(10) if x % 2 == 0]

# Generator expression (round brackets)
gen = (x*x for x in range(5))
print(next(gen))        # 0 (ek-ek value milegi)
print(next(gen))        # 1
```

> **List vs Generator:** List Comprehension pura result ek saath memory me store karta hai. Generator Expression values ko on-demand generate karta hai — bade data ke liye memory-efficient hai.

4.8 Paradigms — Context Manager

Context Manager (with statement) resources (jaise file) ko automatically open aur close karne ke liye use hota hai.

```
with open("data.txt", "w") as f:
    f.write("Hello Python")
# file automatically close ho jayegi, chahe error aaye ya na aaye

# Custom context manager
class MyContext:
    def __enter__(self):
        print("Enter ho rahe hain")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Exit ho rahe hain")

with MyContext():
    print("Andar kaam ho raha hai")
```

5 Tooling — Package Managers, Environments & Code Quality

5.1 Package Managers

Package managers Python libraries ko install, update aur manage karne ke liye use hote hain.

Tool	Kaam
pip (PyPI)	Sabse common — pip install <package>
conda	Data science ke liye popular, non-Python dependencies bhi manage karta hai
poetry	Modern tool — dependency + packaging + virtualenv sab ek jagah

```
pip install requests          # package install karna
pip install requests==2.31.0  # specific version install karna
pip uninstall requests        # package remove karna
pip freeze > requirements.txt # sab installed packages list karna

# Poetry
poetry init          # naya project setup
poetry add requests  # dependency add karna
```

5.2 Environments (Virtual Environments)

Virtual Environment ek isolated space hota hai jisme har project ke apne packages alag se install hote hain, taaki projects ke dependencies aapas me clash na karein.

Tool	Kaam
venv	Python builtin virtual environment tool
virtualenv	venv jaisa hi, thoda purana aur extra features
pyenv	Multiple Python versions manage karne ke liye
Pipenv	pip + virtualenv ek saath (Pipfile use karta hai)
uv	Naya, bahut fast package + environment manager (Rust based)
pyproject.toml	Modern project configuration file (dependencies, build settings)

```
python -m venv myenv          # environment banana
source myenv/bin/activate     # activate karna (Linux/Mac)
myenv\Scripts\activate        # activate karna (Windows)
pip install pandas            # ab is environment me install hoga
deactivate                     # environment se bahar aana
```

5.3 Static Typing

Python dynamically typed hai, lekin type hints se hum variables/functions ka expected type likh sakte hain — isse bugs jaldi pakde jaate hain.

```
def add(a: int, b: int) -> int:
    return a + b

name: str = "Amit"
age: int = 21

# Pydantic - data validation ke liye
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

user = User(name="Riya", age=20)   # agar type galat ho to error dega
```

Tool	Kaam
Pydantic	Runtime par data validate karna type hints se
mypy	Static type checker — code run kiye bina type errors dhundta hai
pyright	Microsoft ka fast type checker (VS Code me use hota hai)
pyre	Facebook/Meta ka type checker

5.4 Code Formatting

Code formatters automatically code ko consistent style me arrange karte hain.

Tool	Kaam
black	Opinionated auto-formatter — code ko fix style me daalta hai
ruff	Bahut fast linter + formatter (Rust based), black se bhi tez
yapf	Google ka configurable code formatter

```
black my_script.py     # code ko auto-format karega
ruff check my_script.py    # linting errors dhundega
ruff format my_script.py   # formatting karega
```

5.5 Documentation

Sphinx ek popular tool hai jo Python code ke docstrings se professional documentation (HTML/PDF) automatically generate karta hai.

```
def add(a, b):
    """
```

```
    Do numbers ko add karta hai.

    Args:
        a (int): pehla number
        b (int): dusra number

    Returns:
        int: dono ka sum
    """
    return a + b
```

5.6 Testing

Testing se hum verify karte hain ki code sahi kaam kar raha hai ya nahi, bina manually check kiye.

Tool	Kaam
doctest	Docstring ke andar likhe examples ko hi test ki tarah run karta hai
unittest / pyUnit	Python ka builtin testing framework
pytest	Sabse popular, simple aur powerful testing framework
nose	Purana test-runner (ab kam use hota hai)
tox	Multiple Python versions/environments me tests automate karta hai

```
# test_math.py (pytest style)
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

# terminal me run karna:
# pytest test_math.py
```

6 Advanced — Frameworks, Concurrency & Asynchrony

6.1 Frameworks — Synchronous

Synchronous frameworks har request ko ek-ek karke, order me process karte hain.

Framework	Use
Pyramid	Flexible, general-purpose web framework
Plotly Dash	Data visualization / analytics dashboards banane ke liye

6.2 Frameworks — Asynchronous

Asynchronous frameworks ek saath multiple requests ko bina block hue handle kar sakte hain — high performance ke liye.

Framework	Use
gevent	Coroutine based networking library
aiohttp	Async HTTP client/server framework
Tornado	Async web framework, real-time apps ke liye achha
Sanic	Bahut fast async web framework

6.3 Frameworks — Sync + Async dono

Framework	Use
FastAPI	Modern, bahut fast, type-hints based API framework
Django	Full-featured 'batteries included' web framework
Flask	Lightweight, simple micro-framework

```
# FastAPI example
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello, FastAPI!"}

# Flask example
from flask import Flask
app = Flask(__name__)
```

```
@app.route("/")
def home():
    return "Hello, Flask!"
```

6.4 Concurrency

Concurrency ka matlab hai multiple tasks ko 'ek saath' handle karna (zaroori nahi ki bilkul parallel ho).

GIL (Global Interpreter Lock)

GIL Python ka ek lock hai jo ek time par sirf ek thread ko Python bytecode execute karne deta hai — isliye CPU-heavy tasks me threading utni fayda nahi deti.

Threading

Threading multiple threads (lightweight tasks) chalata hai — I/O-bound kaam (jaise file/network operations) ke liye achha hai.

```
import threading
import time

def task(name):
    print(f"{name} start")
    time.sleep(2)
    print(f"{name} done")

t1 = threading.Thread(target=task, args=("Thread-1",))
t2 = threading.Thread(target=task, args=("Thread-2",))
t1.start()
t2.start()
t1.join()
t2.join()
```

Multiprocessing

Multiprocessing alag-alag processes chalata hai (har process ka apna Python interpreter aur memory hota hai) — CPU-heavy tasks ke liye achha hai kyunki GIL ka effect nahi padta.

```
from multiprocessing import Process

def square(n):
    print(n * n)

if __name__ == "__main__":
    p1 = Process(target=square, args=(5,))
    p2 = Process(target=square, args=(10,))
    p1.start()
    p2.start()
    p1.join()
    p2.join()
```

6.5 Asynchrony (async/await)

Asyncio se ek hi thread me multiple tasks 'cooperatively' chalte hain — jab ek task wait kar raha ho (jaise network response), tab dusra task chal sakta hai.

```
import asyncio
```

```
async def task(name, delay):
    print(f"{name} start")
    await asyncio.sleep(delay)   # non-blocking wait
    print(f"{name} done")

async def main():
    await asyncio.gather(
        task("Task-1", 2),
        task("Task-2", 1),
    )

asyncio.run(main())
```

Concept	Best for
Threading	I/O-bound (file, network, DB calls)
Multiprocessing	CPU-bound (heavy calculations)
Asyncio	Bahut saare I/O tasks ek saath, single thread me

📝 Quick Revision Sheet

Poore roadmap ka ek-line summary — revision ke liye:

Topic	Ek line me yaad rakhein
Variables	Naam se value store karna, type dynamically decide hota hai
Conditionals	if / elif / else se decision lena
Functions	def se reusable code block banana
List/Tuple/Set/Dict	mutable-ordered / immutable-ordered / unique-unordered / key-value
Loops	for (fixed range) / while (condition tak)
Recursion	Function khud ko call kare, base case zaroori
Sorting	Bubble/Selection/Insertion = O(n²), Merge/Quick = O(n log n)
OOP	Class = blueprint, Object = uska instance
Decorator	Function ko bina change kiye extra behaviour dena
Generator	Values ek-ek karke (lazy) generate karta hai, memory bachata hai
Context Manager	with statement — resource auto open/close
Virtual Env	Har project ke dependencies alag rakhna
GIL	Ek time par ek thread hi Python code chalata hai
Threading	I/O-bound kaam ke liye
Multiprocessing	CPU-bound kaam ke liye
Asyncio	Single thread me bahut saare I/O tasks

> **Final Tip:** Ye notes ek baar padhne ke baad, har topic ka code khud se ek baar likh kar chalayein — practice se hi concepts pakke hote hain. All the best! 🚀